

ConcurrentQueue Collection Class in C# with Examples

In this article, I am going to discuss the **ConcurrentQueue Collection Class in C#** with Examples. Please read our previous article where we discussed [ConcurrentDictionary Collection Class in C#](#) with Examples. At the end of this article, you will understand the following pointers.

1. [What is ConcurrentQueue<T> Class in C#?](#)
2. [Why do we need ConcurrentQueue<T> Collection Class in C#?](#)
3. [Generic Queue Example with Single Thread in C#](#)
4. [Generic Queue Example with Multi-Thread in C#](#)
5. [Generic Queue with Locking Mechanism in C#](#)
6. [ConcurrentQueue Example with more than one Thread in C#](#)
7. [How to Create a ConcurrentQueue<T> Collection in C#?](#)
8. [How to Add Elements into a ConcurrentQueue<T> Collection in C#?](#)
9. [How to access a ConcurrentQueue Collection in C#?](#)
10. [How to Remove Elements from ConcurrentQueue<T> Collection in C#?](#)
11. [How to Get the First Element from the ConcurrentQueue in C#?](#)
12. [How to Copy a ConcurrentQueue Collection to an Existing Array in C#?](#)
13. [ConcurrentQueue<T> Collection Class with Complex Types in C#](#)

What is ConcurrentQueue<T> Class in C#?

The ConcurrentQueue<T> is a Thread-Safe Collection Class in C#. It was introduced as part of C# 4.0 and it belongs to **System.Collections.Concurrent** namespace. It provides a Thread-Safe First-In-First-Out (FIFO) data structure. That means we need to go for ConcurrentQueue<T> Collection when we need First in First Out (FIFO) access to items of a collection in a multi-threaded environment with thread safety.

The working of the ConcurrentQueue<T> is very much similar to the working of the Generic Queue<T> collection class. The only difference between them is that Generic Queue<T> is not thread-safe whereas the ConcurrentQueue<T> is thread-safe. So, we can use Queue<T> class instead of ConcurrentQueue<T> class with multiple threads, but in that case, as a developer, we need to use locks explicitly to provide thread safety which is always time-consuming and error-prone. So, the ideal choice is to use ConcurrentQueue<T> instead of Queue<T> in a multi-threaded environment, and with ConcurrentQueue<T>, as a developer, we don't require to implement any locking mechanism.

Why do we need ConcurrentQueue<T> Collection Class in C#?

Let us understand why we need ConcurrentQueue collection class in C#. So, what we will do here is first we will see the example using Generic Queue, then we will see the thread-safety problem with Generic Queue and how we can resolve the problem by implementing the locking mechanism, and finally, we will how to use ConcurrentQueue collection.

Generic Queue Example with Single Thread in C#:

In the following example, we created a generic queue **MobileOrders** to store order information. Further, if you notice in the below code, the GetOrders method is called from the TestQueue Method in a regular synchronous way. And from the main method, we simply call the TestQueue method.

```

using System;
using System.Collections.Generic;
using System.Threading;
namespace ConcurrentQueueDemo
{
    class Program
    {
        static void Main()
        {
            TestQueue();
            Console.ReadKey();
        }
        public static void TestQueue()
        {
            var MobileOrders = new Queue<string>();
            GetOrders("Pranaya", MobileOrders);
            GetOrders("Anurag", MobileOrders);
            foreach (var mobileOrder in MobileOrders)
            {
                Console.WriteLine($"Order: {mobileOrder}");
            }
        }
        private static void GetOrders(string custName, Queue<string> MobileOrders)
        {
            for (int i = 0; i < 3; i++)
            {
                Thread.Sleep(100);
                string order = string.Format($"{custName} Needs {i+3} Mobiles");
                MobileOrders.Enqueue(order);
            }
        }
    }
}

```

Output:

```

Order: Pranaya Needs 3 Mobiles
Order: Pranaya Needs 4 Mobiles
Order: Pranaya Needs 5 Mobiles
Order: Anurag Needs 3 Mobiles
Order: Anurag Needs 4 Mobiles
Order: Anurag Needs 5 Mobiles

```

As the GetOrders method is called in a synchronous way, the output is also printed similarly i.e. first Pranaya and then Anurag which is what you can see in the above output.

Generic Queue Example with Multi-Thread in C#:

Now, let's modify the previous example to make it async. For that, we have used a Task that will call GetOrders by two different threads. And we have done these changes inside the TestQueue method. Further, we have changed the loop count to 100 inside the GetOrders method and removed the Thread.Sleep statement as shown in the below example.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
namespace ConcurrentQueueDemo
{
    class Program
    {
        static void Main()
        {
            TestQueue();
            Console.ReadKey();
        }
        public static void TestQueue()
        {
            var MobileOrders = new Queue<string>();
            Task t1 = Task.Run(() => GetOrders("Pranaya", MobileOrders));
            Task t2 = Task.Run(() => GetOrders("Anurag", MobileOrders));
            Task.WaitAll(t1, t2); //Wait till both the task completed

            foreach (var mobileOrder in MobileOrders)
            {
                Console.WriteLine($"Order: {mobileOrder}");
            }
        }
        private static void GetOrders(string custName, Queue<string> MobileOrders)
        {
            for (int i = 0; i < 100; i++)
            {
                //Thread.Sleep(100);
                string order = string.Format($"{custName} Needs {i+3} Mobiles');
                MobileOrders.Enqueue(order);
            }
        }
    }
}
```

Output:

```

public static void TestQueue()
{
    var MobileOrders = new Queue<string>();
    Task t1 = Task.Run(() => GetOrders("Pranaya", MobileOrders));
    Task t2 = Task.Run(() => GetOrders("Anurag", MobileOrders));
    Task.WaitAll(t1, t2); //Wait till both the task completed ❌

    foreach (var mobileOrder
    {
        Console.WriteLine($"
    }

private static void GetOrder
{
    for (int i = 0; i < 100;
    {
        //Thread.Sleep(100);
        string order = string.Format($"{custName} Needs {i+3} Mobiles");
        MobileOrders.Enqueue(order);
    }
}

```

Exception Unhandled

System.AggregateException: 'One or more errors occurred.'

Inner Exception

ArgumentException: Destination array was not long enough. Check destIndex and length, and the array's lower bounds.

[View Details](#) | [Copy Details](#)

▸ Exception Settings

You will not get the above exception every time. Try to run the application multiple times and at some time, you will get the above exception.

Why do we get the above exception?

This is because the Enqueue method of Generic Queue<T> Collection Class is not designed to work with more than one thread parallelly i.e. it is not thread-safety. So, Multi-Threading with Generic Queue<T> is unpredictable. It may work in some instances but if you try several times, you are likely to get an exception.

Generic Queue with Locking Mechanism in C#:

In the below example, we use the famous lock keyword to the statement i.e. the statement which adds the order to the queue.

```

using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
namespace ConcurrentQueueDemo
{
    class Program
    {
        static object lockObj = new object();
        static void Main()
        {
            TestQueue();
            Console.ReadKey();
        }
    }
}

```

```

    }
    public static void TestQueue()
    {
        var MobileOrders = new Queue<string>();
        Task t1 = Task.Run(() => GetOrders("Pranaya", MobileOrders));
        Task t2 = Task.Run(() => GetOrders("Anurag", MobileOrders));
        Task.WaitAll(t1, t2); //Wait till both the task completed

        foreach (var mobileOrder in MobileOrders)
        {
            Console.WriteLine($"Order: {mobileOrder}");
        }
    }
    private static void GetOrders(string custName, Queue<string> MobileOrders)
    {
        for (int i = 0; i < 100; i++)
        {
            //Thread.Sleep(100);
            string order = string.Format($"{custName} Needs {i+3} Mobiles'
            lock (lockObj)
            {
                MobileOrders.Enqueue(order);
            }
        }
    }
}

```

Now, run the above code and you will not get any exception. That is fine. So, no exception after putting a lock on the Enqueue method. But what about if the Enqueue is called multiple times, would you use the lock statement everywhere? If you do, then it is a time-consuming process and error-prone as you might forget to use a lock somewhere. The solution is to use ConcurrentQueue.

ConcurrentQueue with more than one Thread in C#:

ConcurrentQueue provides the thread-safety automatically in a multi-threaded environment. Let us rewrite the same example using the ConcurrentQueue class and see the output and then we will discuss the ConcurrentQueue collection class in detail. In the below example, we simply replace the Queue class with ConcurrentQueue. And remove the statement used for locking. Please note the ConcurrentQueue class belongs to System.Collections.Concurrent namespace, so include that namespace.

```

using System;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;
namespace ConcurrentQueueDemo
{
    class Program
    {
        static void Main()
        {

```

```

        TestQueue();
        Console.ReadKey();
    }
    public static void TestQueue()
    {
        var MobileOrders = new ConcurrentQueue<string>();
        Task t1 = Task.Run(() => GetOrders("Pranaya", MobileOrders));
        Task t2 = Task.Run(() => GetOrders("Anurag", MobileOrders));
        Task.WaitAll(t1, t2); //Wait till both the task completed

        foreach (var mobileOrder in MobileOrders)
        {
            Console.WriteLine($"Order: {mobileOrder}");
        }
    }
    private static void GetOrders(string custName, ConcurrentQueue<string>
MobileOrders)
    {
        for (int i = 0; i < 3; i++)
        {
            Thread.Sleep(100);
            string order = string.Format($"{custName} Needs {i+3} Mobiles'
MobileOrders.Enqueue(order);
        }
    }
}

```

Output:

```

Order: Pranaya Needs 3 Mobiles
Order: Anurag Needs 3 Mobiles
Order: Anurag Needs 4 Mobiles
Order: Pranaya Needs 4 Mobiles
Order: Pranaya Needs 5 Mobiles
Order: Anurag Needs 5 Mobiles

```

Now, I hope you understand the basic need for ConcurrentQueue collection class.

Methods, Properties, and Constructors of ConcurrentQueue Class in C#:

Let us understand the different Methods, Properties, and Constructors of ConcurrentQueue Collection Class in C#. If you right-click on the ConcurrentQueue class and select go to definition, then you will see the following. The ConcurrentQueue class belongs to System.Collections.Concurrent namespace and implements the IProducerConsumerCollection<T>, IEnumerable<T>, IEnumerable, ICollection, IReadOnlyCollection<T> interfaces.

```

namespace System.Collections.Concurrent
{
    ...public class ConcurrentQueue<T> : IProducerConsumerCollection<T>, IEnumerable<T>, IEnumerable, ICollection, IReadOnlyCollection<T>
    {
        ...public ConcurrentQueue();
        ...public ConcurrentQueue(IEnumerable<T> collection);

        ...public bool IsEmpty { get; }
        ...public int Count { get; }

        ...public void CopyTo(T[] array, int index);
        ...public void Enqueue(T item);
        ...public IEnumerator<T> GetEnumerator();
        ...public T[] ToArray();
        ...public bool TryDequeue(out T result);
        ...public bool TryPeek(out T result);
    }
}

```

How to Create a ConcurrentQueue<T> Collection in C#?

The ConcurrentQueue<T> Collection Class in C# provides the following two constructors to create an instance of the ConcurrentQueue <T> class.

1. **ConcurrentQueue():** It is used to initialize a new instance of the ConcurrentQueue class.
2. **ConcurrentQueue(IEnumerable<T> collection):** It is used to initialize a new instance of the ConcurrentQueue class that contains elements copied from the specified collection.

Let's see how to create an instance of ConcurrentQueue using the ConcurrentQueue() constructor:

Step1:

As the ConcurrentQueue<T> class belongs to System.Collections.Concurrent namespace, so first, we need to include System.Collections.Concurrent namespace in our program is as follows:

using System.Collections.Concurrent;

Step2:

Next, we need to create an instance of the ConcurrentQueue class using the ConcurrentQueue() constructor as follows:

ConcurrentQueue<type> ConcurrentQueue_Name = new ConcurrentQueue<type>();

Here, the type can be any built-in data type like int, double, string, etc., or any user-defined data type like Customer, Employee, Product, etc.

How to Add Elements into a ConcurrentQueue<T> Collection in C#?

If you want to add elements to a ConcurrentQueue collection in C#, then you need to use the following Enqueue() method of the ConcurrentQueue<T> class.

1. **Enqueue(T item):** The Enqueue(T item) method is used to add an element at the end of the ConcurrentQueue. Here, the parameter item specifies the object to add to the end of the ConcurrentQueue. The value can be null for a reference type i.e. when T is a reference type.

For Example,

ConcurrentQueue<int> queue= new ConcurrentQueue<int>();

The above statement will create a ConcurrentQueue of integer types. So, here we can only add integer-type elements on the ConcurrentQueue. If we try to add anything other than an integer then we will get a compile-time error.

queue.Enqueue(10);

```
queue.Enqueue(20);
queue.Enqueue("Hello"); //Compile-Time Error
```

Note: We cannot add elements into a ConcurrentQueue using Collection Initializer.

How to access a ConcurrentQueue Collection in C#?

We can access all the elements of the ConcurrentQueue collection in C# using a for each loop as follows.

```
foreach (var item in queue)
{
    Console.WriteLine(item);
}
```

Example to Understand How to Create a ConcurrentQueue and Add Elements in C#:

For a better understanding of how to create a ConcurrentQueue, how to add elements, and how to access all the elements from ConcurrentQueue in C# using a for-each loop, please have a look at the following example which shows the above three things.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentQueueDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a ConcurrentQueue to Store Integer Values
            ConcurrentQueue<int> concurrentQueue = new ConcurrentQueue<int>();
            //Adding Elements to ConcurrentQueue using Enqueue Method
            concurrentQueue.Enqueue(10);
            concurrentQueue.Enqueue(20);
            concurrentQueue.Enqueue(30);
            //Adding Duplicate
            concurrentQueue.Enqueue(30);
            //As int is not a Reference type so null can not be accepted by the
            ConcurrentQueue
            //concurrentQueue.Enqueue(null); //Compile-Time Error
            //As the concurrentQueue is integer type, so string values can not
            accepted
            //concurrentQueue.Enqueue("Hello"); //Compile-Time Error
            //Accessing all the Elements of concurrentQueue using For Each Loop
            Console.WriteLine("Concurrent Queue Elements");
            foreach (var item in concurrentQueue)
            {
                Console.WriteLine(item);
            }
            Console.ReadKey();
        }
    }
}
```

```
}
```

Output:

```
Concurrent Queue Elements
10
20
30
30
```

How to Remove Elements from ConcurrentQueue<T> Collection in C#?

In ConcurrentQueue, the Elements which is added first will be the element to be removed first. That means we are allowed to remove elements from the beginning of the ConcurrentQueue. The ConcurrentQueue Collection Class in C# provides the following TryDequeue method to remove an element.

1. **TryDequeue(out T result):** This method tries to remove and return the object at the beginning of the concurrent queue. Here, the parameter result contains the object removed if the operation was successful. If no object was available to be removed, the value is unspecified. This method returns true if an element was removed and returned from the beginning of the ConcurrentQueue successfully; otherwise, false.

Let us see an example to understand the TryDequeue(out T result) methods of ConcurrentQueue<T> Collection Class in C#. Please have a look at the following example which shows the use of the TryDequeue method.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentQueueDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a ConcurrentQueue to Store Integer Values
            ConcurrentQueue<int> concurrentQueue = new ConcurrentQueue<int>();
            //Adding Elements to ConcurrentQueue using Enqueue Method
            concurrentQueue.Enqueue(10);
            concurrentQueue.Enqueue(20);
            concurrentQueue.Enqueue(30);
            concurrentQueue.Enqueue(40);
            Console.WriteLine("All Concurrent Queue Elements Before Removing");
            foreach (var item in concurrentQueue)
            {
                Console.WriteLine(item);
            }
            bool IsRemoved = concurrentQueue.TryDequeue(out int Result);
            Console.WriteLine($"\\nTryDequeue Return : {IsRemoved}");
            Console.WriteLine($"TryDequeue Result Value : {Result}");
        }
    }
}
```

```

        Console.WriteLine("\nAll Concurrent Queue Elements After Removing'
foreach (var item in concurrentQueue)
{
    Console.WriteLine(item);
}

Console.ReadKey();
    }
}
}

```

Output:

```

All Concurrent Queue Elements Before Removing
10
20
30
40

TryDequeue Return : True
TryDequeue Result Value : 10

All Concurrent Queue Elements After Removing
20
30
40

```

How to Get the First Element from the ConcurrentQueue in C#?

The ConcurrentQueue<T> Collection Class in C# provides the following two methods to get the first element of the ConcurrentQueue collection.

1. **TryDequeue(out T result):** This method tries to remove and return the object at the beginning of the concurrent queue. Here, the parameter result contains the object removed if the operation was successful. If no object was available to be removed, the value is unspecified. This method returns true if an element was removed and returned from the beginning of the ConcurrentQueue successfully; otherwise, false.
2. **TryPeek(out T result):** This method tries to return an object from the beginning of the ConcurrentQueue without removing it. Here, the parameter result contains an object from the beginning of the ConcurrentQueue or an unspecified value if the operation failed. It returns true if an object was returned successfully; otherwise, false.

For a better understanding, please have a look at the below example which shows how to get the first element from the ConcurrentQueue using the TryDequeue(out T result) and TryPeek(out T result) methods of ConcurrentQueue<T> Collection Class in C#.

```

using System;
using System.Collections.Concurrent;
namespace ConcurrentQueueDemo

```

```

{
    class Program
    {
        static void Main()
        {
            //Creating a ConcurrentQueue to Store Integer Values
            ConcurrentQueue<int> concurrentQueue = new ConcurrentQueue<int>();
            //Adding Elements to ConcurrentQueue using Enqueue Method
            concurrentQueue.Enqueue(10);
            concurrentQueue.Enqueue(20);
            concurrentQueue.Enqueue(30);
            concurrentQueue.Enqueue(40);
            //Accesing all the Elements of ConcurrentQueue using For Each Loop
            Console.WriteLine($"All Concurrent Queue Elements Count:
{concurrentQueue.Count}");
            foreach (var item in concurrentQueue)
            {
                Console.WriteLine(item);
            }
            // Removing and Returning the First Element from ConcurrentQueue u
TryDequeue method
            bool IsRemoved = concurrentQueue.TryDequeue(out int Result1);
            Console.WriteLine($"\\nTryDequeue Return : {IsRemoved}");
            Console.WriteLine($"TryDequeue Result Value : {Result1}");
            //Printing Elements After Removing the First Element
            Console.WriteLine($"All Concurrent Queue Elements After TryDequeue
{concurrentQueue.Count}");
            foreach (var element in concurrentQueue)
            {
                Console.WriteLine($"{element} ");
            }
            // Removing and Returning the First Element from ConcurrentQueue u
TryDequeue method
            bool IsPeeked = concurrentQueue.TryPeek(out int Result2);
            Console.WriteLine($"\\nTryPeek Return : {IsPeeked}");
            Console.WriteLine($"TryPeek Result Value : {Result2}");
            //Printing Elements After Peek the First Element
            Console.WriteLine($"All Concurrent Queue Elements After TryPeek: (
{concurrentQueue.Count}");
            foreach (var element in concurrentQueue)
            {
                Console.WriteLine($"{element} ");
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```
All Concurrent Queue Elements Count: 4
10
20
30
40

TryDequeue Return : True
TryDequeue Result Value : 10
All Concurrent Queue Elements After TryDequeue: Count 3
20
30
40

TryPeek Return : True
TryPeek Result Value : 20
All Concurrent Queue Elements After TryPeek: Count 3
20
30
40
```

How to Copy a ConcurrentQueue Collection to an Existing Array in C#?

In order to copy a ConcurrentQueue Collection to an Existing Array in C#, we need to use the following CopyTo method of the ConcurrentQueue Collection Class.

1. **CopyTo(T[] array, int index):** This method is used to copy the ConcurrentQueue Elements to an existing one-dimensional Array, starting at the specified array index. Here, the parameter array specifies the one-dimensional array that is the destination of the elements copied from the ConcurrentQueue. The Array must have zero-based indexing. The index parameter specifies the zero-based index in the array at which copying begins.

This method works on one-dimensional arrays and does not change the state of the ConcurrentQueue. The elements are ordered in the array in the same way as the order of the elements from the beginning of the ConcurrentQueue to the end. Let us see an example for a better understanding of the CopyTo(T[] array, int index) method of the ConcurrentQueue<T> Collection Class in C#.

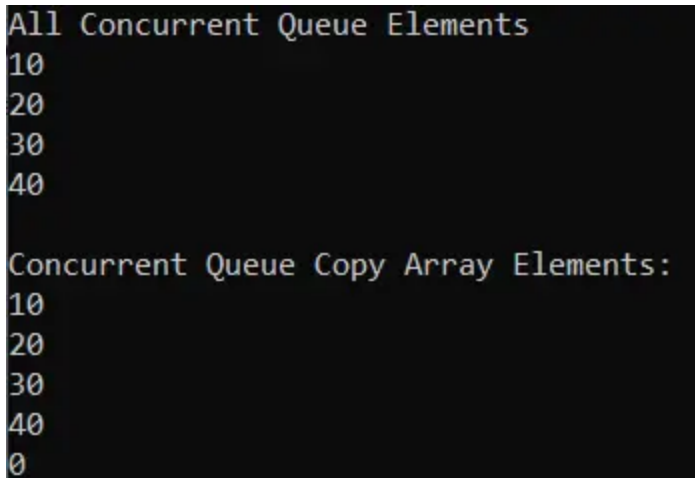
```
using System;
using System.Collections.Concurrent;
namespace ConcurrentQueueDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a ConcurrentQueue to Store Integer Values
            ConcurrentQueue<int> concurrentQueue = new ConcurrentQueue<int>();
            //Adding Elements to ConcurrentQueue using Enqueue Method
            concurrentQueue.Enqueue(10);
            concurrentQueue.Enqueue(20);
```

```

        concurrentQueue.Enqueue(30);
        concurrentQueue.Enqueue(40);
        //Accessing all the Elements of ConcurrentQueue using For Each Loop
        Console.WriteLine($"All Concurrent Queue Elements");
        foreach (var item in concurrentQueue)
        {
            Console.WriteLine(item);
        }
        //Copying the ConcurrentQueue to an array
        int[] concurrentQueueCopy = new int[5];
        concurrentQueue.CopyTo(concurrentQueueCopy, 0);
        Console.WriteLine("\nConcurrent Queue Copy Array Elements:");
        foreach (var item in concurrentQueueCopy)
        {
            Console.WriteLine(item);
        }
        Console.ReadKey();
    }
}

```

Output:



```

All Concurrent Queue Elements
10
20
30
40

Concurrent Queue Copy Array Elements:
10
20
30
40
0

```

ConcurrentQueue<T> Collection Class with Complex Types in C#.

As of now, we have used the ConcurrentQueue Collection class with Primitive Data Types such as int, double, etc. Now, let us see how to use the ConcurrentQueue Collection with complex types such as Employee, Student, Customer, Product, etc. For a better understanding, please have a look at the below example where we use the ConcurrentQueue<T> Collection with the user-defined Student type.

```

using System;
using System.Collections.Concurrent;
namespace ConcurrentQueueDemo
{
    class Program
    {

```

```

static void Main()
{
    //Creating a ConcurrentQueue to Store Integer Values
    ConcurrentQueue<Student> concurrentQueue = new ConcurrentQueue<Student>();
    //Adding Elements to ConcurrentQueue using Enqueue Method
    concurrentQueue.Enqueue(new Student() { ID = 101, Name = "Anurag",
"CSE" });
    concurrentQueue.Enqueue(new Student() { ID = 102, Name = "Mohanty",
"CSE" });
    concurrentQueue.Enqueue(new Student() { ID = 103, Name = "Sambit",
"ETC" });
    //Accessing all the Elements of ConcurrentQueue using For Each Loop
    Console.WriteLine($"All Concurrent Queue Elements");
    foreach (var item in concurrentQueue)
    {
        Console.WriteLine($"ID: {item.ID}, Name: {item.Name}, Branch:
{item.Branch}");
    }

    Console.ReadKey();
}
}
public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}
}

```

Output:

```

All Concurrent Queue Elements
ID: 101, Name: Anurag, Branch: CSE
ID: 102, Name: Mohanty, Branch: CSE
ID: 103, Name: Sambit, Branch: ETC

```

In the next article, I am going to discuss the [ConcurrentStack<T> Collection Class in C#](#) with Examples. Here, in this article, I try to explain the **ConcurrentQueue<T> Collection Class in C#** with Examples. I hope this ConcurrentQueue Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.